

Context Free Grammars (CFG)

- For Regular languages we design a Regular Expression. But, for non-Regular language we can't design FA.

- CFG is used to write Regular grammar for Regular and non-Regular language.

- CFG is a machine used to construct a FA for regular & non Regular lang.

eg 1) $a^n b^n$... Here we can't keep record of 'a's' so, above is non Regular language

2) Prime Numbers

a^p ... p is prime no's. but they are not fix.

Above things require to store something at someplace!

- RL we can simply write RE but, for non-Regular language we would not write FA because we have to maintain Record in count of something with count to another thing. That's the reason we can't write designed FA.

- From above solve the problem using CFG. is basically for RL & NRE

$$RL + NRE = CFG$$

- First we write Reg CFG for regular and non Regular language both, then we design PDA

CFG: CFG stand for Context free Grammar
 - It is a formal grammar which is used to generate all possible pattern of strings in a given formal language.
 - Context free grammar G can be defined by four tuples. $G = (V, T, P, S)$:

G : is the Grammar.

V : is a set of non terminal symbols.

T : is a Terminal symbols. $a, b, c, \dots$

P : is a set of production values

S : is a Start symbol.

egs: Construct the CFG for the language having any number of a 's over the set $Z = \{a\}$
 soln: 0

Regular Expression R.E. = a^*
 production Rule for RE is

$S \rightarrow aS$ Rule 1

$S \rightarrow \epsilon$ Rule 2

If we want to derive a string 'aaaaa' we can start with start symbols.

S

aS

aaS

Rule 1

$aaas$

Rule 1

$aaaaS$

Rule 1

$aaaaaS$

Rule 1

$aaaaaaS$

Rule 1

$aaaaaa\epsilon$

Rule 2

$aaaaaa$

eg2: construct a CFG for the regular expression $(0+1)^*$

soluⁿ:

$$L = \{ \epsilon, 0, 1, 00, 11, 01, 10, \dots \}$$

CFG can be given by production rule (P)

$$S \rightarrow 0S \mid 1S$$

$$S \rightarrow \epsilon$$

The rules are in the combination of 0's and 1's with the start symbol. since $(0+1)^*$ indicates $\{ \epsilon, 0, 1, 01, 10, 00, 11, \dots \}$

In this set, ϵ is a string so in the rule we can set the rule $S \rightarrow$

eg3: construct a CFG for a language $L = \{ w^R w \mid w \in (a,b)^+ \}$.

Solution: The string that can be generated for a given language is $\{ aacaa, bcb, abcb, baab, abbcba, \dots \}$

The grammar could be

$$S \rightarrow aSa \quad \text{Rule 1}$$

$$S \rightarrow bSb \quad \text{Rule 2}$$

$$S \rightarrow \epsilon \quad \text{Rule 3}$$

Now, if we want to derive a string 'abbcba', we can start with start symbols.

$$S \rightarrow aSa$$

$$S \rightarrow abSba \quad \text{From Rule 2}$$

$S \rightarrow abbCbba \dots$ From Rule 3

eg4:

construct a CFG for the language
 $L = a^n b^{2n}$ where $n \geq 1$

Solution:

The string that can be generated
a given language is $\{abb, aabbbb, aaabbbbbb\}$

The grammar could be:

$S \rightarrow aSbb \mid abb$

now, if we want to derive a string "aabbbb"
we can start with start symbols.

$S \rightarrow aSbb$

$S \rightarrow aabbbb$

* Derivation: For string derivation we have two types.

1) Sentential form

2) Derivation.

Derivation is sequence of production Rules.
It is used to get the i/p string through
these production Rules.

During parsing, we have to take 2 decisions.
These are as follows.

1) We have to decide the non-terminal which
is to be replaced.

2) We have to decide the production
Rules by which the non-terminal will be
replaced.

Leftmost Derivation:

In the leftmost derivation, the input is scanned and replaced with the production rule from left to right. So, in leftmost derivation we read the input string from left to right.

Rightmost Derivation:-

In Rightmost derivation the i/p is scanned and replaced with the production rules from right to left. So, in rightmost derivation we read the i/p string from right to left.

Leftmost Derivation examples.

eg: production Rules $E \rightarrow E + E$, $E \rightarrow E - E$
 $E \rightarrow a/b$

Show processing of above String from mentioned production Rules $a - b + a$

The leftmost derivation is:-

$E \rightarrow E + E$	} using LMD.
$E \rightarrow E - E + E$	
$E \rightarrow a - E + E$	
$E \rightarrow a - b + E$	
$E \rightarrow a - b + a$	

Rightmost derivation examples

eg: Production Rules $E \rightarrow E + E$, $E \rightarrow E - E$, $E \rightarrow a/b$

Show processing of above string from mentioned production rules $a - b + a$

The rightmost derivation is.

$E \rightarrow E - E$
 $E \rightarrow E - E + E$
 $E \rightarrow E - E + a$
 $E \rightarrow E - b + a$
 $E \rightarrow a - b + a$

} using RMD.

eg 3: Derive string "aabbabba" for leftmost derivation & rightmost derivation using
 CFG: $S \rightarrow aB \mid bA$, $A \rightarrow a \mid aS \mid bAA$
 $B \rightarrow b \mid bS \mid aBB$

Soln:

Leftmost derivation

S
 aB $S \rightarrow aB$
 $aaBB$ $B \rightarrow aBB$
 $aabb$ $B \rightarrow b$
 $aaabbS$ $B \rightarrow bS$
 $aabbab$ $S \rightarrow aB$
 $aaabbabS$ $B \rightarrow bS$
 $aaabbabba$ $S \rightarrow bA$
 $aabbabba$ $A \rightarrow a$

Rightmost Derivation

S
 aB $S \rightarrow aB$
 $aaBB$ $B \rightarrow aBB$
 $aaabbS$ $B \rightarrow bS$
 $aaabbba$ $S \rightarrow bA$
 $aaabbba$ $A \rightarrow a$
 $aaabbba$ $B \rightarrow bS$
 $aaabbba$ $S \rightarrow bA$
 $aaabbba$ $A \rightarrow a$

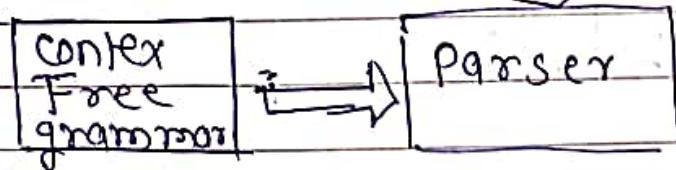
eg4. Derive the string '00101' for leftmost derivation and rightmost derivation using a CFG : $S \rightarrow AIB$, $A \rightarrow 0A|E$, $B \rightarrow 0B|IB|E$

Solution:

leftmost Derivation 00101	Rightmost Derivation 00101
S	S
AIB	AIB
0AIB	AIOB
00AIB	AIOIB
00IB	AIOI
0010B	0AIOI
0010IB	00AIOI
00101	00101

* parse Tree :-

stream of Tokens



$E \rightarrow E + E \mid E * E \mid id$

Rule:

1. $E \rightarrow E + E$
2. $E \rightarrow E * E$
3. $E \rightarrow id$

$= (NTPS)$

$N = \{E\}$

$T = \{+, *, id\}$

$P = \{ \{E \rightarrow E + E\} \{E \rightarrow E * E\} \{E \rightarrow id\} \}$

$S = E$

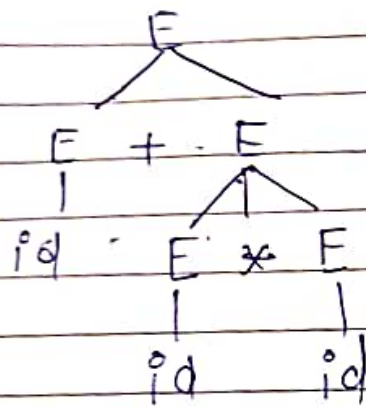
Left Most Derivation

$$\begin{aligned}
 E &= E + E \\
 &= id + E \\
 &= id + E * E \\
 &= id + id * E \\
 &= id + id * id
 \end{aligned}$$

Right Most Derivation

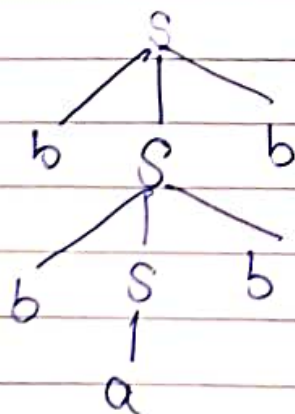
$$\begin{aligned}
 E &\Rightarrow E + E \\
 &= E + E * E \\
 &= E + E * id \\
 &= E + id * id \\
 &= id + id * id
 \end{aligned}$$

Parse Tree Derivation



eg: Draw a derivation tree for the string "bbabb" for the C.F.G. given by

$$S \rightarrow bSb \mid a \mid b$$

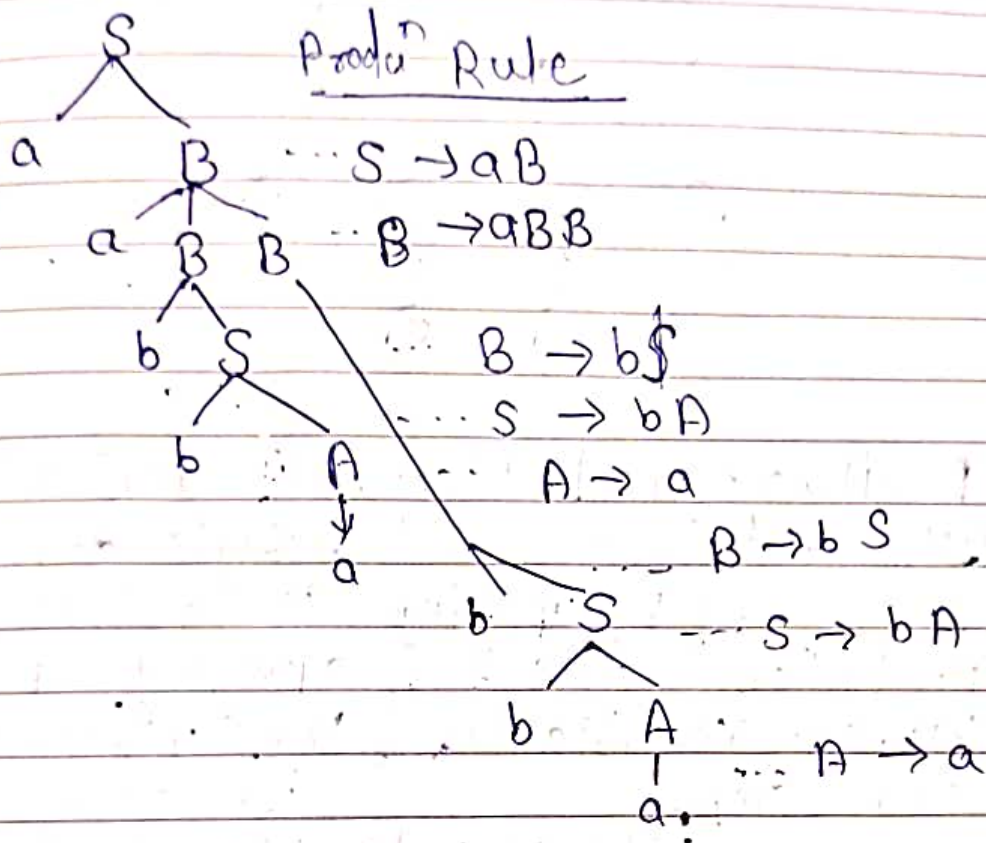


start From left to Right
 $\Rightarrow$ bbabb

eg2: construct a derivation tree in the string.

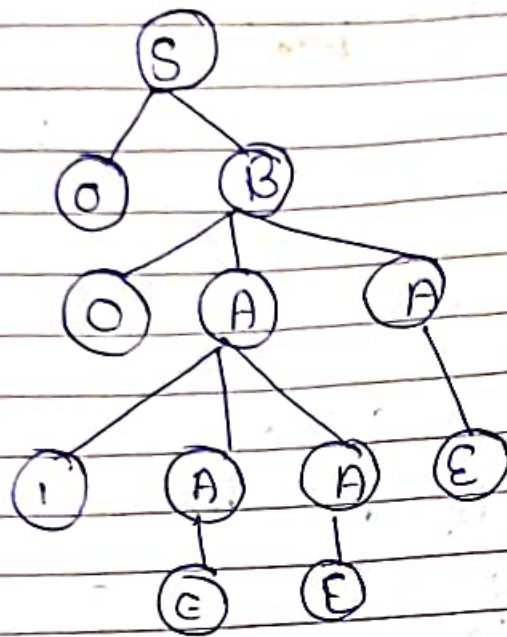
aabbabba in C.F.G. $S \rightarrow aB \mid bA$
 $A \rightarrow a \mid aS \mid bAA$
 $B \rightarrow b \mid bS \mid aBB$

Leftmost Derivation



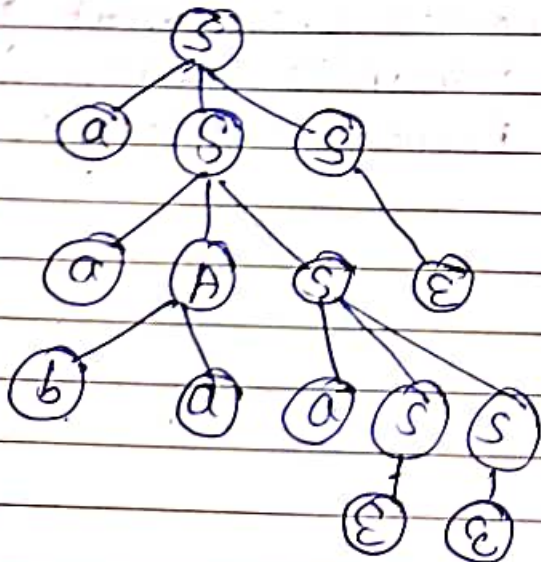
Note: A Derivation tree or parse tree is an ordered rooted tree that graphically represent the semantic information of strings derived from a Context free grammar.

eg: For the grammar $G = \{V, T, P, S\}$
 where $S \rightarrow OB, A \rightarrow 1AA | \epsilon, B \rightarrow OA$

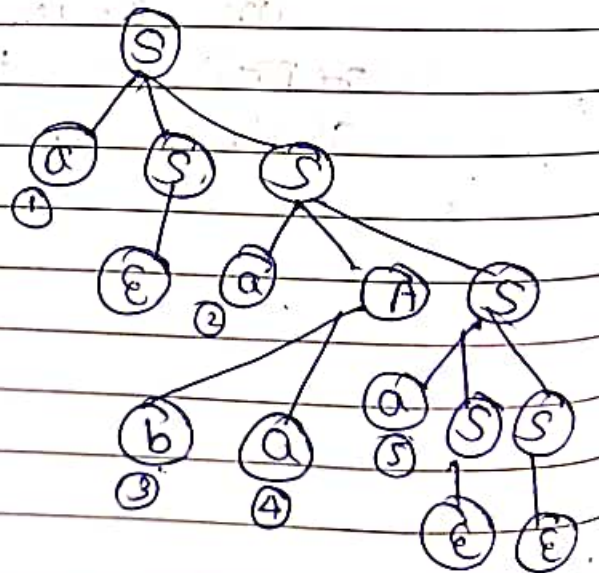


* Leftmost and Rightmost derivation Tree
 Leftmost derivation Tree is obtained by applying production to the leftmost variable in each step.
 Rightmost derivation Tree is obtained by applying production to the rightmost variable in each step.

eg: generate string "aabaa" from A
 $S \rightarrow aAS | aSS | \epsilon, A \rightarrow SbA | ba$



"aabaa"



"aabaa"

* Application of context-free Grammars :-

Context free grammars were originally conceived by N. Chomsky as a way to describe Natural language. But that promise has not been fulfilled. But, CFG's as a way to describe some instances of these concept.

1) Grammars are used to describe programming language. There is a mechanical way of turning the language description as a CFG into a parser. The component of the compiler that discovers the structure of the source program and represents that structure by a parse tree. So, this is one of the first way in which theoretical ideas in Computer Science found their way into practice.

2) Parser: Many aspect of Programming language have a structure that may be described by regular expressions.

In programming language use paranthesis, repeat the paranthesis, rules, balanced string, if..else condition some simple test done by parser where context free grammar is used.

3) The YACC parser-Generator :- (Yet another compiler-compiler)
Input to YACC is a context free grammar. In this notation that differs only in details from the one we have used here. Associated with each production is an

on action, which is a fragment of 'c' code that is performed whenever a node of the parse tree corresponds to this production is created. Typically, the action is code to construct that node, although in some YACC applications the tree is not actually constructed, the action does something else such as emit a piece of object code.

4) Mark-up language :-

CFG that describes as much of the structure of the HTML language which covers some aspects of HTML do require the power of CFG's. For instances, each pair of tags that are a corresponding beginning and ending pair eg... $\langle EM \rangle$ & $\langle /EM \rangle$.

5) XML & document type definition.

XML ie. extensible Markup language where CFG play a more vital role. As a part of process of using that language.

DTD [Document type definition] is essentially a context-free grammar, with its own notation for describing the variables & productions.

DTT Represent How many times the child & node should appear & according child is ordered.

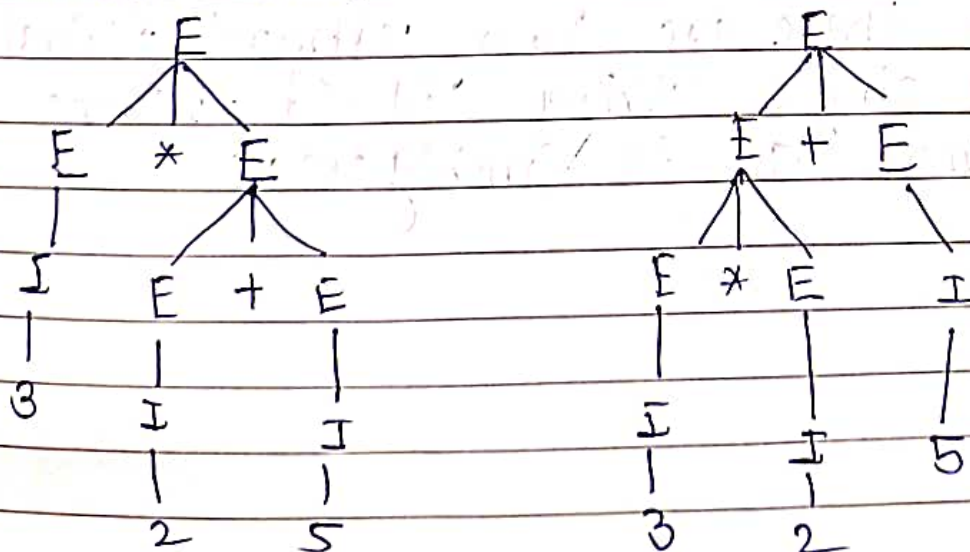
* Ambiguity in Grammars and Languages :-

A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given input string. If the grammar is not ambiguous so it is called unambiguous grammar.

If the grammar is ambiguous grammar then, it is not good for compiler construction. No method can automatically detect to remove ambiguity, but we can remove ambiguity by re-writing the whole grammar without ambiguity.

eg 1: $E \rightarrow I$ $E \rightarrow E + E$ $E \rightarrow E * E$
 $E \rightarrow (E)$ $I \rightarrow \epsilon | 0 | 1 | 2 | \dots | 9$

For the string $3 * 2 + 5$ the derivation & Parse tree.



Since there are two parse tree for a single string $3 * 2 + 5$, the grammar G is ambiguous.

Page No. _____
Date _____

eg 2: check whether the given grammar is ambiguous or not. $[id + id - id]$

$E \rightarrow E + E$ $E \rightarrow E - E$
 $E \rightarrow id$

solution: From above grammar string $id + id - id$ can be derived in 2 ways
First leftmost Derivation

$E \rightarrow E + E$
 $\rightarrow id + E$
 $\rightarrow id + E - E$
 $\rightarrow id + id - E$
 $\rightarrow id + id - id$

second leftmost derivation

$E \rightarrow E - E$
 $\rightarrow E + E - E$
 $\rightarrow id + E - E$
 $\rightarrow id + id - E$
 $\rightarrow id + id - id$

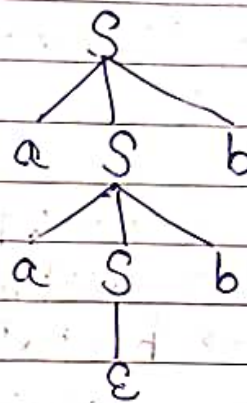
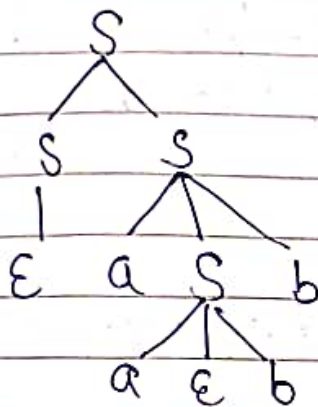
Since there are two leftmost derivation for a single string ' $id + id - id$ ' the grammar G is ambiguous.

eg3. check whether the given grammar G is ambiguous or not.

$$S \rightarrow aSb / SS$$

$$S \rightarrow \epsilon$$

Solution: For the string aabb the above grammar can generate two parse tree.



since there are two parse tree for a single string 'aabb'. The grammar G is ambiguous.

eg4: check whether the given grammar G is ambiguous or not.

$$A \rightarrow AA$$

$$A \rightarrow (A)$$

$$A \rightarrow a$$

Solution Two parse tree is form the grammar is ambiguous.

